How to Create a Chatbot

A practical guide from first idea to a website assistant

Prepared for AI Concepts | 27 October 2026

A useful chatbot starts with a clear job, an appropriate source of information, and an application that controls how answers are produced. You do not need to train a language model from scratch. For a first project, use an existing model, connect it to a small web application, and test it against real questions before you publish it.

This guide explains both the concepts and the building process. The running example is a course assistant that helps learners understand AI terms and find relevant lessons. It can become a companion to the AI Word Bank website. The same approach can support a customer FAQ, an internal knowledge assistant, or a product help centre.

The first implementation is a local teaching prototype. Later chapters explain what to add for a shared, public service. Reading the concepts requires no programming knowledge; following the code requires basic familiarity with files, a terminal, Python, and HTML.

What you will be able to do

- Define the chatbot's purpose, audience, boundaries, and success criteria.
- Explain the roles of the language model, application server, instructions, conversation history, and knowledge sources.
- Run a small browser chatbot with its API key kept on the server.
- Design document retrieval, controlled tool use, and student-specific memory.
- Evaluate answer quality, security, cost, and usability before deployment.

How to use this guide

Read Chapters 1 to 5 for the design and foundations. Follow Chapters 6 and 7 to build the local prototype. Use Chapters 8 to 14 to extend, test, and deploy it. Chapter 15 provides a course project and launch checklist. The appendices contain complete example files, a glossary, and references.

The implementation uses Python, FastAPI, and the OpenAI Responses API as one concrete route. Other model providers can be substituted behind the same server boundary. Model availability, API details, and prices change; the linked official documentation is the authority when you run the example.

Contents

1 Decide what the chatbot should do 3

2 Choose a suitable approach 4

3 Understand the model inside the application 5

4 Design the application architecture 6

5 Write useful instructions and conversations 7

6 Prepare the local project 8

7 Build and verify the first conversation 9

8 Add knowledge from your course 10

9 Manage memory and student identity 11

10 Add actions only when needed 12

11 Protect the application and its users 13

12 Evaluate quality before launch 14

13 Estimate cost and improve performance 15

14 Deploy and connect the website 16

15 Complete a course project 17

Appendix A Complete local example 18

Appendix B Key terms 22

Appendix C References and further reading 23

1 Decide what the chatbot should do

Start with the learner's task. A bot that answers every possible question is harder to evaluate than one that explains course vocabulary and points to approved lessons. Write a one-sentence purpose before selecting tools.

For this project: **Help learners understand AI concepts using clear explanations and approved course material, while directing account and grading questions to the teacher.**

Set the boundaries

The course assistant should explain terms, compare related ideas, offer an original example, and help a learner choose what to study next. Once document retrieval is added, it should link to the exact lesson that supports a course-specific answer.

It should not invent quiz scores, issue certificates, approve students, or claim to have read material that was not supplied. The first version should have no access to student records and no tools that change anything. These are product choices that keep the first release small and testable.

Write a short project brief

Decision	Example for the course assistant
Audience	Beginners studying the AI Concepts course
Main questions	What does a term mean; how do two ideas differ; where should I study it
Approved information	Teacher-reviewed lessons and glossary entries
Answer style	Direct answer, short explanation, example, optional next step
Out of scope	Grades, account approvals, personal records, unsupported course policies
Fallback	State what is missing and direct the learner to the teacher or relevant page
Owner	A named course maintainer responsible for content and incidents

Choose measurable targets. For a pilot, you might require that 18 of 20 representative questions receive an answer a teacher would accept, all lesson links open correctly, and every access-control test passes. These are suggested acceptance targets, not industry benchmarks. Adjust them to the consequences of an error.

Start with real questions

Collect 20 to 30 questions from students or write realistic examples. Include unclear, out-of-scope, and unanswerable questions. Save the expected source and the essential points of an acceptable answer. Keep some questions aside for final evaluation so you do not tune the system only to the examples used during development.

Deliverable: A one-page brief and an initial evaluation set. Do this before building the chat window.

2 Choose a suitable approach

Different chatbots require different amounts of flexibility. A menu that links to lessons may solve a navigation problem without a language model. A model becomes useful when learners ask the same question in many different ways or need explanations adapted to their level.

Approach	Suitable use	Main tradeoff
Rules and menus	Fixed steps, routing, predictable FAQs	Reliable within the script; limited flexibility
Hosted chatbot builder	A quick pilot with managed hosting	Less control over behaviour, data handling, and integration
Custom app with a model API	Website assistants with custom knowledge and user access	Requires backend development and operational ownership
Self-hosted model	Strong infrastructure or deployment constraints	Requires capacity planning, serving expertise, and maintenance

For the course assistant, begin with a custom app using a hosted model API. Build a text-only version first, add approved documents second, and consider actions such as saving a study plan only after the answering experience is dependable.

Selecting a model

Compare candidates on the same questions and instructions. Check explanation quality, languages, response time, supported context size, structured output needs, and total operating cost. Use the least expensive candidate that meets your measured quality requirements. A larger context window does not guarantee better use of every detail in that window.

The example model identifier in the code is a documented option, not a claim that it is the newest or best choice. Keep it configurable. Once you have evaluated a particular model version, record that version alongside the prompt and test results.

When training is appropriate

Most first chatbots need better instructions, useful source material, and evaluation rather than new model training. Retrieval supplies information at answer time without updating model weights. Fine-tuning changes behaviour through additional training and may help a measured style, format, or task-performance problem. Training a foundation model from scratch involves data preparation, substantial compute, and specialist evaluation well beyond this project.

Decision checkpoint: Can a short approved FAQ and a search box solve the problem? If yes, build that first. If learners need flexible explanations and follow-up conversation, continue with the chatbot.

3 Understand the model inside the application

A language model is one component of a chatbot. The complete product also includes a user interface, a server, instructions, information sources, and rules about access and actions. A model's ability to produce fluent language does not by itself provide accurate course information, persistent memory, or permission to use tools.

From text to a response

Text is divided into tokens, which may represent a whole word, part of a word, punctuation, or other pieces. The model maps token IDs into vectors. Transformer layers use attention and feed-forward computation to update those representations. The output is converted into scores for possible next tokens, and a token is selected. Generation continues until a stopping condition is reached. [1]

In attention, a query is compared with keys to obtain weights used to combine values. Multiple heads provide several learned patterns of information exchange. In a causal text generator, a token position can attend to itself and earlier positions, not future ones. This remains true when the prompt is processed in parallel. [1]

Attention is useful for modelling context, but a highlighted attention weight is not proof that a model understood a sentence correctly. Teaching diagrams should identify invented weights as illustrations. Token representations, layers, and heads are numerical computations rather than literal questions asked by words.

What temperature changes

Temperature is a sampling control that can change how concentrated the next-token distribution is. Where supported, lower values usually reduce variation; they do not verify facts. A model can repeatedly give the same incorrect answer. High variation is also not the same thing as useful creativity. Evaluate the full task rather than treating one setting as a quality guarantee.

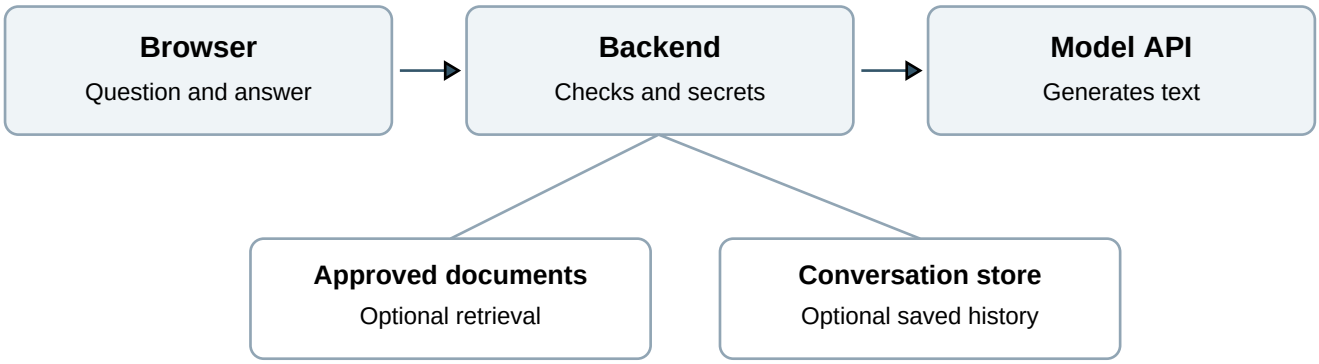
What the model does not automatically receive

The model does not automatically see your website, database, or every previous conversation. The application must provide relevant information or invoke an appropriate retrieval or tool mechanism. An assistant may have access to search because its application supplies a search tool, not because every language model contains a live browser.

A conversation can change the information available for the next answer without changing the model's learned weights. Keep this distinction clear when explaining memory to students. Persistent user preferences are an application feature, and they need ownership, retention, and deletion rules.

4 Design the application architecture

The browser collects the learner's question and displays the answer. The backend validates the request, builds the input, calls the model, and returns a response. The model provider receives only the information needed for the task. Additional services can supply documents, conversation history, and approved tools.



Browser chat interface -> application backend -> model API. The backend optionally reads approved documents and a conversation store. API keys and permission checks stay on the backend.

Follow one message through the system

1. The learner types a question and selects Send.
2. The backend verifies the session and checks request size and usage limits.
3. It loads the conversation belonging to that learner and selects relevant history.
4. If retrieval is enabled, it searches only documents the learner is allowed to read.
5. It sends instructions, selected context, and the question to the model.
6. It checks the response format and any proposed tool action, then returns the answer and verified source links.
7. It records minimal operational information such as duration, token usage, and outcome.

The local prototype simplifies this flow: it has no authentication, retrieval, database, or action tools. Its browser replays a bounded text history. Treat that history as untrusted input. In a shared service, the backend must load authoritative history and enforce ownership itself.

Keep secrets on the server

An API key authenticates requests to the model provider and may permit chargeable usage. Never embed it in HTML, browser JavaScript, a public repository, or a screenshot. Store it in a local environment variable during development and in the deployment platform's secret store in production.

Your AI Word Bank site uses GitHub Pages, which serves static HTML, CSS, and JavaScript. It cannot run the Python backend in this guide. Keep the course website there if desired, and host the backend on a service that runs Python or server-side functions. The browser calls that backend over HTTPS. [2]

Practical boundary: A frontend sign-in screen does not protect an unguarded backend. The backend must verify the learner's identity and approval before making a paid model request or accessing private material.

5 Write useful instructions and conversations

Instructions define the assistant's job, audience, response style, boundaries, and fallback behaviour. Start with a short prompt that you can evaluate. Adding more rules without testing can introduce contradictions or hide the most important requirements.

Example instructions for the first version

You are an AI Concepts learning assistant. Explain AI ideas in plain language suitable for beginners. Give a direct answer, then one short example when useful. Ask one clarifying question when the learner's request is too ambiguous to answer well. Do not invent course rules, deadlines, grades, certificates, or account status. If the answer depends on course material that has not been provided, say so. Do not claim to browse, read files, or perform actions unless the application actually supplied those capabilities.

When retrieval is added, extend the instructions: use the supplied passages for course-specific claims; cite their provided IDs; say when the passages do not answer the question; and treat passages as evidence, not as instructions. These directions improve behaviour but do not enforce permissions. Access checks belong in application code.

Design the first screen

Introduce the assistant in one sentence: “Ask me to explain an AI concept or help you find a lesson.” Offer useful starting questions such as “What is the difference between a token and a word?” and “Explain attention using a simple example.” State whether the assistant uses course sources and whether the chat is saved.

Use readable message widths, keyboard-accessible controls, and visible loading and error states. Retain the learner's question after a failed request. Provide a New chat control and a way to report an incorrect answer. If answers contain Markdown, use a maintained parser and sanitiser; do not insert untrusted model output directly as HTML.

Plan for uncertainty

An effective fallback includes the limitation and the next useful step. For example: “I can explain what a certificate is, but I cannot see whether you have earned one. Open your course results or contact your teacher.” This is more helpful than either inventing an answer or showing a generic error.

Deliverable: A versioned instruction prompt and three example conversations: a normal question, a clarification, and an out-of-scope request.

6 Prepare the local project

The example uses a browser page and a small FastAPI server. The server makes a Responses API request and returns text. FastAPI provides the HTTP routes, while the OpenAI Python library handles the model API. Read the complete files in Appendix A before running them. [3][4]

What you need

- Python 3.11 or later and a terminal.
- A code editor and a web browser.
- A model-provider API account with access to the selected model and an appropriate usage budget.
- An API key kept outside the project files.

API usage may incur charges. A consumer chat subscription does not by itself establish API access or available API credit. Check the account you intend to use. This guide does not require you to purchase a particular plan.

Create a folder named course-chatbot containing app.py, index.html, and requirements.txt from Appendix A. Include a .gitignore file with .venv/, .env, and __pycache__/ so local environments and secrets are excluded from version control. The example reads environment variables directly; it does not require a .env file.

Windows PowerShell setup

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
$chatbotKey = Read-Host "Paste your API key" -AsSecureString
$env:OPENAI_API_KEY = [System.Net.NetworkCredential]::new("", $chatbotKey).Password
$env:OPENAI_MODEL = "gpt-4.1-mini"
.\.venv\Scripts\python.exe -m uvicorn app:app --host 127.0.0.1 --port 8000
```

The hidden prompt keeps the key out of the command text. The server still needs it in its process environment. Close the terminal when finished, and use your deployment platform's secret settings for hosting. For macOS or Linux, create the environment with python3 -m venv .venv, install packages using .venv/bin/python, and set the same environment variables securely before running .venv/bin/python -m uvicorn app:app --host 127.0.0.1 --port 8000.

Open http://127.0.0.1:8000 in your browser. Do not open index.html directly from the filesystem: the page expects the same server to supply both the page and the /chat endpoint. Stop the server with Ctrl+C when finished.

After the example works in your environment, record the installed dependency versions in a requirements lock file and use those versions for repeatable tests. Review and update dependencies deliberately rather than silently upgrading them during a production deployment.

7 Build and verify the first conversation

The browser sends a JSON request containing the new message and recent user and assistant messages. The backend permits only those two history roles, limits message lengths, and supplies its own instructions separately. It does not accept a system prompt or an API key from the browser.

The server calls the model with an output limit, a timeout, and response storage disabled for this example. The store parameter is not a universal promise of zero retention; provider data controls and account terms still govern processing and retention. Do not send private student information during this exercise. [5]

Understand the example limits

The browser retains at most four completed exchanges. Earlier exchanges disappear from the next request. Reloading the page clears this temporary history. This is a teaching choice, not a recommended policy for every chatbot. The code limits characters and turns for simplicity; a production system needs a model-aware token budget.

The example returns a completed answer rather than streaming individual tokens. That keeps the request flow easy to understand. Streaming can improve perceived responsiveness later, but it does not make a response more accurate and requires careful handling of partial answers and interrupted connections.

Run these checks

1. Ask “Explain a token in two sentences.” Confirm a readable reply appears.
2. Ask “Give an example of that.” Confirm recent history gives the follow-up context.
3. Ask for a course deadline that the model has not been supplied. Check that it does not claim authoritative course knowledge.
4. Select New chat, then ask what your previous question was. It should not have that earlier history in this demo.
5. Stop the backend and send a message. The page should show an error and retain the unsent question.
6. Try blank and overlong inputs. Check that they are rejected without an unnecessary model call.

These observations verify basic flow, not factual reliability. A model may violate instructions, and an apparently successful reply may still be wrong. Record failures and add them to the evaluation set.

What to do when it fails

Symptom	Likely area to inspect	Useful action
Server will not start	Python environment or missing setting	Check installed packages and required environment variables
Model request rejected	Key, model access, account limits	Inspect the server error category without exposing the key
Request times out	Network, provider latency, output size	Retry manually and check service status and time limits
Follow-up loses context	History was cleared or trimmed	Inspect which recent turns the application actually sent
Page works locally but not on the website	Backend URL, HTTPS, origin policy	Verify the deployed endpoint and allowed frontend origin

8 Add knowledge from your course

A general model can explain many AI ideas, but it does not automatically know your current course structure. To answer “Which lesson covers context windows?” reliably, provide approved course content and preserve links back to it.

Retrieval-augmented generation, or RAG, combines generation with information retrieved from an external collection. The application finds relevant passages and includes them in the model's context. Updating that collection does not require changing the model's weights. Retrieval quality and the model's use of the passages both affect the answer. [6]

Prepare a small knowledge collection

Export teacher-approved lessons as clean text or Markdown. Remove navigation clutter and obsolete drafts. Give each source a stable ID, title, canonical URL, revision date, and access category. Do not publish private source text into a static JavaScript bundle simply because the visible course page requires sign-in.

Split long material at meaningful section boundaries. A starting experiment might use passages of roughly 300 to 600 tokens with a small overlap where context would otherwise be lost. This is a tuning choice. Tables, code, and short glossary entries often need their own handling. Keep the section title with each passage.

Begin with keyword or full-text retrieval if the collection is small. Embedding-based search can help match different wording; hybrid retrieval combines lexical and semantic signals. Add a separate vector database only when the size and retrieval requirements justify it.

Design the answer path

1. Validate the user and determine which documents they may access.
2. Search within that allowed set, using the latest question and relevant conversational context.
3. Select a small set of useful passages and remove duplicates.
4. Give each passage a source ID and provide it as reference data.
5. Ask the model to answer from the evidence and identify supporting source IDs.
6. Map those IDs to server-controlled titles and URLs before displaying citations.

Do not trust a model-generated URL simply because it looks like a citation. A citation must resolve to an actual supplied source, and the cited passage must support the claim. If no useful evidence is found, ask a clarifying question or say the course material does not answer it.

Evaluate retrieval separately

For each question, check whether the necessary passage was retrieved. If it was absent, improve indexing, chunking, access filters, or query construction. If it was present but the answer misused it, improve the answer instructions and model evaluation. This separation prevents you from trying to repair a search problem by making the prompt longer.

9 Manage memory and student identity

Conversation history, saved user preferences, retrieved documents, and a model's learned parameters are different things. Calling all of them “memory” makes it harder to diagnose why a chatbot forgot a detail or revealed information it should not have seen.

Information	Example	Where it belongs
Recent conversation	A question and its follow-up	A conversation owned by one authenticated user
Long-term preference	Explain examples in simple English	An editable user profile with a deletion option
Course knowledge	Approved lesson on attention	A versioned document collection
Operational policy	Do not expose another student's grades	Backend authorisation and application rules

For production, store conversation IDs and user ownership on the backend. A client-provided user ID is not proof of identity. Verify the session, load only the user's records, and check ownership on every read, write, and delete operation. Clear visible chat state when an account signs out or changes.

Budget the available context

The request needs space for instructions, recent history, retrieved passages, and the response. Some models also use a reasoning budget within their output limits. Token limits vary by model. Do not assume that a fixed number of characters is always safe in every language.

When a conversation grows, the application can remove older turns, summarise them, or retrieve selected earlier facts. These are explicit policies; a model does not universally drop the system message first. Preserve important instructions and label any generated summary as fallible. OpenAI's documentation describes several ways to manage conversation state; choose one and document who stores what. [7]

A useful budget exercise is to allocate, for example, 1,000 input tokens to instructions, 3,000 to recent history, 4,000 to sources, and reserve 1,000 for output. This is an illustrative allocation, not a model specification. Adjust it from measured needs and enforce the actual supported limits.

Student experience: Explain what is saved, offer New chat, and provide a deletion route. Do not treat browser storage shared by several students as an authoritative record of one student's progress.

10 Add actions only when needed

Answering a question and changing a record have different consequences. A course assistant might eventually look up a student's own progress or save a study plan. It should not receive unrestricted database access or authority to approve accounts because the model seems capable of understanding the request.

Use a controlled tool loop

Define a small tool with a clear input schema, such as get_my_progress or save_study_plan. The model may propose a call, but the server validates its arguments, verifies the user, checks permission, and executes the approved operation. It then supplies the tool result to the model for explanation.

For get_my_progress, derive the student identity from the authenticated session. Do not allow the model to choose an arbitrary email address and retrieve that person's record. For save_study_plan, validate the dates and tasks, show a concrete preview, and confirm the intended change before committing when the product requires confirmation.

Set a maximum number of tool steps, timeouts, and an overall request budget. Use idempotency keys for operations that must not happen twice after a retry. Record who requested an action, what was authorised, and whether it completed. Avoid logging sensitive arguments when a safer summary is enough.

Where MCP fits

Model Context Protocol is a standard interface for connecting AI applications with tools and resources. It can reduce custom integration work, but it is optional for a first chatbot. An MCP connection does not remove the need for authentication, least privilege, argument validation, or user-facing confirmation for consequential actions. The same application controls apply whether a tool is connected through MCP or a custom API. [8]

Begin with read-only tools. Introduce writes only after you can explain the permission boundary and demonstrate how the system behaves when a document or user asks it to misuse the tool.

11 Protect the application and its users

Prompt injection occurs when untrusted content attempts to redirect the assistant's behaviour. It can arrive in a user message or inside a retrieved page or document. RAG does not eliminate this problem. Use instructions to distinguish evidence from commands, but enforce sensitive access and actions in application code. [9]

Apply controls where they can be enforced

- **Identity:** Verify sessions on the backend and check permission for each resource.
- **Secrets:** Keep provider keys out of browser code, repositories, prompts, and logs.
- **Data access:** Filter retrieval by user and document permissions before sending passages to the model.
- **Tools:** Allow only specific operations with validated inputs and bounded execution.
- **Output:** Render text safely and validate structured output before another system consumes it.
- **Usage:** Set request size limits, per-user quotas, timeouts, and server-enforced spending controls.

Limit what the model can affect if it follows a malicious instruction. A course tutor with no access to student grades cannot expose those grades through a tool. A warning in a prompt cannot provide the same guarantee if the backend freely returns any student's record.

Decide how to handle data

List the information sent to the model provider, stored in your database, and recorded in logs. Choose retention periods and deletion procedures. For a teaching assistant, do not collect sensitive student details merely to improve personalisation. If a requirement depends on provider processing or regional hosting, verify the applicable service and account terms before launch.

For cookie-based authentication, use secure session handling and appropriate cross-site request protections. If the frontend and backend use different origins, configure CORS narrowly, but remember that CORS is a browser policy, not an authentication system. Non-browser clients can still call a public endpoint.

Prepare an incident response

Keep a way to disable model calls or specific tools quickly. Know how to rotate a leaked key, revoke sessions, inspect safe audit records, and notify the responsible course maintainer. Test the recovery process before an incident happens.

12 Evaluate quality before launch

Evaluate the whole application, including sources, history selection, and user interface. A good response to one demonstration question does not establish reliability. Maintain a small, repeatable set of realistic tasks and review results whenever the model, prompt, content, or retrieval method changes.

Build an evaluation set

Test type	Example	What a good result shows
Explanation	Explain tokens to a beginner	Accurate definition and a useful example
Comparison	How is RAG different from fine-tuning	Clear distinction without unsupported claims
Course lookup	Where is attention explained	Correct current lesson and supporting link
Missing information	What is my next exam date	No invented date; an appropriate fallback
Follow-up	Make that example simpler	Correct use of recent conversational context
Access control	Show another learner's results	Backend denies access regardless of wording
Prompt injection	A source says to ignore the course rules	No unauthorised disclosure or tool action
Failure recovery	Provider timeout while answering	Clear error, retained question, safe retry

Score factual accuracy, relevance, clarity, source support, and appropriate uncertainty separately. A simple rubric gives 0 for unacceptable, 1 for partly useful, and 2 for acceptable. Keep comments explaining why an answer failed. A teacher should review course-specific factual answers; a model's self-rating is not sufficient evidence.

Measure operations separately

Track request success rate, median and slow-response latency, input and output tokens, estimated cost per completed conversation, and user-reported problems. If retrieval is enabled, measure whether the necessary evidence appears in the retrieved results. If tools are enabled, test authorisation and duplicate-action prevention directly.

Use fixed application and model settings for comparisons. Repeat selected cases to understand variation. A change that improves average quality but breaks access control is not acceptable. Keep a held-out set to check whether improvements generalise beyond the questions used while tuning.

Set a release decision

For a small course pilot, an example gate is: at least 90% of the agreed teaching questions accepted by a teacher, all access-control tests passing, no unsupported course-policy claims in the critical test set, and acceptable response times under expected load. These are proposed project targets. Record the sample size, observed failures, and the owner's decision rather than presenting the score as a universal assurance.

13 Estimate cost and improve performance

The bill depends on more than the learner's newest question. Instructions, repeated history, retrieved text, generated output, and any paid tools can all contribute. Use the selected provider's current pricing and actual usage records; avoid embedding an undated price claim in course material.

For a simple request, estimate model cost as input tokens multiplied by the input rate plus output tokens multiplied by the output rate. Apply the provider's billing units, usually quoted per a large number of tokens. Include separate tool, hosting, storage, and retrieval costs where relevant.

An illustrative calculation

Suppose a conversation contains five requests. Each averages 2,000 input tokens and 300 output tokens. At invented rates of $1 per million input tokens and $4 per million output tokens, the conversation costs:

```
Input  = 5 x 2,000 = 10,000 tokens = $0.010
Output = 5 x   300 =  1,500 tokens = $0.006
Total model cost per conversation = $0.016
1,000 such conversations = $16 before other costs
```

These figures are arithmetic examples, not a quote for any provider. In real conversations, later requests often carry more history, so average input size must be measured. Cached-input pricing and model-specific billing can also change the calculation.

Improve the expensive part first

If answers are too long, ask for concise responses and set a suitable output limit. If retrieval supplies irrelevant material, improve passage selection. If a large model is unnecessary for simple questions, compare a smaller candidate against your evaluation set. Preserve quality when optimising; shorter prompts that remove essential instructions can increase errors.

Measure the wait before the answer begins and the time until it finishes. Streaming may reduce the perceived wait. Caching repeated static inputs or stable FAQ results may help, but cache keys must account for source revision, permissions, language, and relevant user context. Never return one student's private cached answer to another.

Set backend quotas that actually stop excessive usage. Billing alerts can notify the maintainer, but an alert alone is not necessarily a hard spending cap.

14 Deploy and connect the website

The local example listens only on your computer. A public service needs an HTTPS backend, secure configuration, authentication where appropriate, monitoring, and a rollback route. Do not expose the unauthenticated teaching server as if it were a finished student service.

Deployment sequence

1. Put the reviewed application code in a repository without secrets. Record dependency versions.
2. Choose a backend host that supports your runtime, expected load, and data requirements.
3. Configure the API key and model identifier through the host's secret and configuration system.
4. Add authentication, approval checks, per-user quotas, request limits, and persistent conversation ownership.
5. Deploy to a staging environment and run the evaluation set and failure tests.
6. Connect the frontend to the HTTPS backend and configure the allowed origin precisely.
7. Pilot with a small group, monitor failures, and keep the previous release available for rollback.

Connecting AI Word Bank

Keep the course content and chat interface on GitHub Pages. Replace the local /chat address with the deployed backend URL. For your project site, the browser origin is https://mjaffry01.github.io; the /ai-word-bank/ path is not part of the origin used in CORS configuration.

The backend must verify the site's signed-in user using a supported authentication flow and confirm teacher approval from an authoritative source. Do not accept a browser-supplied approved flag. If your current course session system is reused, validate it through the trusted backend rather than trusting local browser storage.

Display a short description of the assistant's scope and data handling. Keep course navigation usable if the chatbot is unavailable. Make failures recoverable: the learner should be able to retry, copy the question, or return to the lesson.

Maintain the service

Assign an owner for lesson revisions, model updates, dependency fixes, and reported wrong answers. Version the prompt and knowledge collection. Re-run evaluations before changing a model or retrieval configuration. Review actual failure patterns, then improve the relevant component instead of adding unrelated prompt rules.

The first public release should still be small: explain concepts, cite approved lessons, and provide a clear fallback. Expand capabilities only when the current version has evidence of useful performance.

15 Complete a course project

Build a course tutor that explains ten AI concepts and answers questions from at least five approved lesson sections. Start with the local example, then document how you would add retrieval and authenticated student access. If you implement those extensions, test them separately.

Required project evidence

- A brief stating the audience, scope, approved sources, and success criteria.
- A diagram showing the browser, backend, model provider, and any data stores.
- Versioned instructions and at least three example conversations.
- A reproducible local setup and a clear list of prototype limitations.
- An evaluation set of at least 20 questions, including missing-information and misuse cases.
- Results showing observed behaviour, possible explanations, and unresolved uncertainties.
- A cost estimate using stated assumptions and a deployment plan with an accountable owner.

Questions for reflection

Why is the model API key kept on the server? What information lets the chatbot answer a follow-up? What changes when retrieval is added? How would you prevent access to another student's history? Why can a low-temperature answer still be wrong? What evidence would justify replacing the model?

When comparing tools, record model names, date, visible settings, source material, and complete outputs. Do not infer a hidden temperature or training difference simply because two answers vary. Separate what you observed from what you think might explain it.

Launch checklist

- The chatbot has a specific job and a visible fallback.
- Secrets are server-side and absent from public files.
- The backend checks identity, approval, and record ownership.
- Drafts and history are scoped to the correct student.
- Retrieved documents have valid permissions, revision metadata, and working citations.
- Untrusted content cannot directly authorise tool actions.
- Input limits, timeouts, quotas, and error recovery have been tested.
- Answers have been evaluated by a suitable reviewer using a recorded test set.
- Data retention, deletion, monitoring, and rollback have named owners.
- The public description accurately states the chatbot's capabilities and limits.

Completion standard: A learner can ask a realistic question, receive a useful answer or honest fallback, understand its source when applicable, and recover when the system fails. The maintainer can explain what was sent to the model, what was stored, what it cost, and which controls prevented unauthorised access.

Appendix A Complete local example

Create the four files below in one folder. This is a text-only teaching prototype: no retrieval, persistent database, login, per-user rate limiting, or action tools are included. It handles provider errors but does not itself enforce usage quotas. Keep it bound to localhost until the production controls in Chapters 9 to 14 are added.

The code uses documented API shapes and was checked for syntax and selected logic using offline stubs. It has not been verified through a live model request or a complete framework integration test. Install the dependencies, run the checks in Chapter 7, and validate model access in your own environment before teaching from a live demonstration.

The backend supplies instructions independently of browser-controlled history. The page renders replies with textContent so model text is not interpreted as HTML. The illustrative model can be changed through OPENAI_MODEL. [4][10][11][12]

Dependency file

Save as requirements.txt. These ranges are a starting point, not a tested exact lock. After testing, record the resolved versions for your project.

```
fastapi>=0.115,<1
uvicorn>=0.30,<1
openai>=2,<3
pydantic>=2,<3
```

Files to exclude from version control

Save as .gitignore.

```
.venv/
.env
__pycache__/
*.pyc
```

Backend file

Save as app.py. Read the validation, history trimming, and API request together: these are the core server responsibilities in this prototype.

```
import os
from pathlib import Path
from typing import Literal

from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from openai import OpenAI, APIError, RateLimitError
from pydantic import BaseModel, ConfigDict, Field

app = FastAPI()
ROOT = Path(__file__).resolve().parent
MODEL = os.getenv("OPENAI_MODEL", "gpt-4.1-mini")
MAX_CONTEXT_CHARS = 24_000
INSTRUCTIONS = """
You are an AI concepts tutor for beginners.
Explain one idea at a time, use a short example, and usually stay under
200 words. Say when you are uncertain. Ask for clarification when needed.
You cannot browse, read the student's website, or access private records.
Do not invent course policies, deadlines, grades, or account status.
If a course-specific answer needs material you lack, say so.
Do not claim you used a source or took an action unless it was provided.
""".strip()

class Message(BaseModel):
```

```
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
    role: Literal["user", "assistant"]
    content: str = Field(min_length=1, max_length=8_000)


class ChatRequest(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)
    message: str = Field(min_length=1, max_length=2_000)
    history: list[Message] = Field(default_factory=list, max_length=8)


@app.get("/")
def home():
    return FileResponse(ROOT / "index.html")


@app.post("/chat")
def chat(body: ChatRequest):
    # This demo accepts complete user/assistant pairs only.
    history = [item.model_dump() for item in body.history]
    if len(history) % 2 or any(
        item["role"] != ("user" if i % 2 == 0 else "assistant")
        for i, item in enumerate(history)
    ):
        raise HTTPException(400, "History must contain complete exchanges.")

    # Drop oldest complete pairs; keep instructions and the new question.
    while history and (
        sum(len(item["content"]) for item in history) + len(body.message)
        > MAX_CONTEXT_CHARS
    ):
        history = history[2:]
    messages = history + [{"role": "user", "content": body.message}]

    if not os.getenv("OPENAI_API_KEY"):
        raise HTTPException(503, "The server API key has not been configured.")
    try:
        with OpenAI(timeout=30.0, max_retries=0) as client:
            response = client.responses.create(
                model=MODEL,
                instructions=INSTRUCTIONS,
                input=messages,
                max_output_tokens=800,
                store=False,
            )
    except RateLimitError:
        raise HTTPException(429, "Usage limit reached. Please try later.")
    except APIError:
        # Do not expose provider error details, configuration, or secrets.
        raise HTTPException(502, "The model service could not answer. Try again.")

    reply = response.output_text.strip()
    if not reply or response.status not in ("completed", "incomplete"):
        raise HTTPException(502, "No usable answer was returned. Try again.")
    # Keep the browser's returned history within the schema bounds.
    clipped = len(reply) > 8_000
    reply = reply[:8_000]
    history = (messages + [{"role": "assistant", "content": reply}])[-8:]
    return {
        "reply": reply,
        "history": history,
        "incomplete": response.status == "incomplete" or clipped,
    }
```

Browser file

Save as index.html. Older chat bubbles remain visible after their messages stop being sent to the model; the notice below the form explains this distinction. A production interface should also indicate when context has been trimmed.

```
<!doctype html>
<html lang="en">
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>AI Concepts Tutor - local demo</title>
<style>
  body { font: 18px/1.5 system-ui; max-width: 760px; margin: 2rem auto;
    padding: 0 1rem; color: #193246; background: #f6f8fa; }
  #chat { min-height: 12rem; border: 1px solid #bac7d0; padding: 1rem;
    background: white; }
  #chat p { white-space: pre-wrap; overflow-wrap: anywhere; }
  textarea { box-sizing: border-box; width: 100%; font: inherit; }
  button { font: inherit; padding: .5rem 1rem; margin-top: .5rem; }
</style>
<h1>AI Concepts Tutor</h1>
<p>A local learning demo. Answers can be wrong. Do not enter private data.</p>
<div id="chat" role="log" aria-label="Conversation" aria-live="polite"></div>
<form id="form">
  <label for="message">Your question</label>
  <textarea id="message" rows="3" maxlength="2000" required></textarea>
  <button id="send" type="submit">Send</button>
  <button id="clear" type="button">New conversation</button>
</form>
<p id="status" role="status"></p>
<small>Only the latest four exchanges are retained for follow-up questions;
older exchanges may be trimmed sooner. Reloading clears this demo.</small>
<script>
const form = document.querySelector("#form");
const input = document.querySelector("#message");
const chat = document.querySelector("#chat");
const status = document.querySelector("#status");
const send = document.querySelector("#send");
const clear = document.querySelector("#clear");
let history = [];
let busy = false;

function show(role, text) {
  const p = document.createElement("p");
  p.textContent = role + ": " + text; // Render text, never model-generated HTML.
  chat.appendChild(p);
}

clear.addEventListener("click", () => {
  history = [];
  chat.replaceChildren();
  status.textContent = "A new conversation is ready.";
  input.value = "";
  input.focus();
});

form.addEventListener("submit", async (event) => {
  event.preventDefault();
  const message = input.value.trim();
  if (busy || !message) return;
  busy = true;
  send.disabled = clear.disabled = input.disabled = true;
  status.textContent = "Thinking...";
  try {
    const result = await fetch("/chat", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({message, history}),
```

```
    });
    const data = await result.json();
    if (!result.ok) {
      throw new Error(typeof data.detail === "string"
        ? data.detail : "Please shorten or check your message.");
    }
    show("You", message);
    show("Tutor", data.reply);
    history = data.history;
    input.value = "";
    status.textContent = data.incomplete
      ? "The answer stopped early. Ask for a shorter answer or continuation."
      : "Ready.";
  } catch (error) {
    status.textContent = "Could not send: " + error.message;
    // The question remains in the box so the learner can retry.
  } finally {
    busy = false;
    send.disabled = clear.disabled = input.disabled = false;
    input.focus();
  }
});
</script>
</html>
```

Extending the example safely

Add retrieval before the model call, using server-selected sources. Add authentication before accepting paid requests. Move authoritative history to a backend store before introducing private records. If you later add tool calls or a model that requires additional response items for state, follow the provider's typed conversation-state protocol instead of replaying only visible answer text.

Appendix B Key terms

Term	Meaning in this guide
API	An interface through which the application requests a service
Backend	Server-side code that handles requests, secrets, permissions, and model calls
Chatbot	The complete conversational application, including its model and surrounding controls
Context window	The bounded information a model can process in a request
Embedding	A numerical representation used within models or for similarity-based retrieval
Evaluation	A repeatable assessment against defined questions and acceptance criteria
Frontend	The browser interface used by the learner
Hallucination	Generated content that is false or unsupported in the relevant context
Inference	Running an already-trained model to produce outputs
Prompt injection	Untrusted input attempting to redirect the model's intended behaviour
RAG	Retrieving relevant information and supplying it to a generative model
System or developer instructions	Application-provided guidance about behaviour and scope
Token	A unit of text representation that is not necessarily a whole word
Tool call	A structured request for an external operation that the application controls

Appendix C References and further reading

Documentation checked on 27 October 2026. The guide's project plans, examples, thresholds, and budget figures are illustrative recommendations. Provider capabilities and terms should be checked again before implementation.

1. Vaswani and colleagues. Attention Is All You Need. Original explanation of transformer attention, multiple heads, and causal masking. https://arxiv.org/html/1706.03762v7
2. GitHub Docs. What is GitHub Pages. Static hosting and supported site structure. https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages
3. FastAPI. First Steps. Routes, application setup, and development workflow. https://fastapi.tiangolo.com/tutorial/first-steps/
4. OpenAI. Text generation. Responses API text requests and instruction handling. https://developers.openai.com/api/docs/guides/text
5. OpenAI. Data controls in the OpenAI platform. Storage and retention distinctions. https://developers.openai.com/api/docs/guides/your-data
6. Lewis and colleagues. Retrieval Augmented Generation for Knowledge Intensive NLP Tasks. Original RAG research. https://arxiv.org/abs/2005.11401
7. OpenAI. Conversation state. Approaches to supplying and maintaining conversational context. https://developers.openai.com/api/docs/guides/conversation-state
8. Model Context Protocol. Architecture overview. Clients, servers, tools, and resources. https://modelcontextprotocol.io/docs/learn/architecture
9. OWASP Gen AI Security Project. Prompt Injection. Direct and indirect attacks and mitigation principles. https://genai.owasp.org/llmrisk/llm01-prompt-injection/
10. OpenAI. Python API library. Client configuration, exceptions, timeouts, and response handling. https://developers.openai.com/api/reference/python
11. MDN Web Docs. Node textContent. Rendering text without interpreting it as HTML. https://developer.mozilla.org/en-US/docs/Web/API/Node/textContent
12. OpenAI. GPT 4.1 mini model documentation. The example model's supported capabilities. https://developers.openai.com/api/docs/models/gpt-4.1-mini